

Introducere în programare - curs și laborator - suport electronic

(C) 2024-2025 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#) | [Material pentru laboratorul de grafica](#)

Laboratorul de grafica

Scopul acestui laborator este familiarizarea studenților cu elemente foarte simple de geometrie analitică, în contextul utilizării unei biblioteci grafice foarte simple. Studenții vor fi capabili să folosească primitiva grafică `line(x1, y1, x2, y2)`, care trasează un segment de dreaptă între două puncte de coordonate `(x1, y1)` și `(x2, y2)`, pentru diferite reprezentări grafice practice, cum ar fi:

- desenarea unei table de șah sau a unei table pentru un joc de masă
- desenarea unui poligon regulat și a unei stele regulate, cu n laturi / colțuri
- desenarea unor figuri definite recursiv, precum fractalii
- trasarea graficului unei funcții oarecare
- trasarea bioritmului unei persoane
- reprezentarea grafică a felului în care se formează o imagine într-o lentilă convergentă

1. O bibliotecă grafică elementară

Scopul nostru este de a realiza diferite exerciții de grafică, în care accentul să-l punem pe partea matematică și de algoritmică și nu pe tehnologie sau pe elemente de programare orientată-obiect (care vor fi predate la alte cursuri). De aceea, vom utiliza o interfață grafică simplă, la care noi putem adăuga propriile funcții mai complexe, create pe baza unor funcții simple predefinite. Așadar, vom folosi cea mai simplă bibliotecă grafică, numită **WinBGIm**, care extinde funcțiile din **Graphics.h**.

Scopul nostru nu este de a folosi biblioteci grafice sofisticate, pentru a face efecte de grafică tridimensională sau jocuri cu un design grafic foarte frumos. Pentru asemenea lucruri, puteți folosi alte biblioteci grafice precum **Dislin**, **SDL**, **SFML** (vezi mai jos!), **OpenGL**, **GLUT** etc. Folosirea acestor biblioteci grafice pentru dezvoltarea de jocuri nu este interzisă, dar fiind mult mai complicate decât **WinBGIm**, nu putem oferi ajutor de niciun fel și rămâne la latitudinea voastră să vă documentați singuri.

Studenții cu experiență în instalarea și utilizarea bibliotecilor grafice **SDL**, **SFML** sau **Dislin** (nu și **OpenGL** sau **GLUT**) sunt încurajați să contacteze pe dl. profesor Bogdan Pătruț pentru include în website-ul laboratorului ghiduri simplificate de instalare a lor pentru CodeBlocks 20 și sistemul de operare Windows.

Atenție! Biblioteca grafică va fi necesară și pentru unele din proiectele din partea a doua a semestrului. **Nota nu crește** dacă proiectul este vizual mult mai frumos, dacă partea de interfață este făcută cu ajutorul unor **componente/obiecte prefabricate**, de genul butoane, casete de selecție, liste simple sau combinate, ferestre de dialog sau alte **elemente GUI predefinite**.

Dimpotrivă, **studenții sunt încurajați să-și definească ei propriile elemente de interfață grafică**, folosind propriile structuri de date și bazându-se pe primitivele grafice elementare puse la dispoziție de o bibliotecă grafică cât mai simplă, cum este **WinBGIm**.

Daca doriți sa instalați CodeBlocks 17.12, la care nu aveți probleme în instalarea bibliotecii grafice WinBGIm, aveți aici link:

<https://sourceforge.net/projects/codeblocks/files/Binaries/17.12/Windows/codeblocks-17.12mingw-setup.exe/download>

1. Dacă veți folosi sistemul de operare Windows, cu CodeBlocks, o versiune mai veche (16-19), împreună cu compilatorul MinGW 32-bit, atunci:

▼ Click aici pentru instrucțiuni de instalare

Descărcați biblioteca grafică WinBGIm de aici: [WINBGIM.ZIP](#)

. Atenție: pe web puteți găsi și alte variante ale acestei arhive, dar e posibil să fie nefuncționale.

Dupa ce descarcati și scoateți fișierele din această arhivă, veți face următoarele lucruri:

1. Extrageți fișierele **graphics.h**, **wingim.h** și **libbgi.a** din arhivă.
2. Copiați fișierele **graphics.h** și **wingim.h** în folderul **include** din folderul compilatorului MinGW, care se găsește, cel mai probabil, aici:
c:\Program Files (x86)\CodeBlocks\MinGW\include
3. Copiați fișierul **libbgi.a** în folderul **lib** al directorului compilatorului MinGW, care se găsește, cel mai probabil, aici:
C:\Program Files (x86)\CodeBlocks\MinGW\lib
4. În Code::Blocks accesați **Settings >> Compiler ... >> linker settings**
5. Faceți click pe butonul **Add** în partea **Link libraries**, apoi răsfoiți și selectați fișierul **libbgi.a**
6. În partea dreaptă (adică la Other linker options) scrieți următoarele comenzi:
-lbgi -lgdi32 -lcomdlg32 -luuid -loleaut32 -lole32
7. Faceți click pe OK.

Următorul videoclip explică foarte clar pașii pe care trebuie să-i urmați.

▼ CLICK AICI

02. How to setup graphics.h in CodeBlocks v17.12 | CodeW...



2. Dacă veți folosi sistemul de operare Windows, cu CodeBlocks, o versiune mai nouă (20), împreună cu compilatorul MinGW 64-bit, atunci:

În acest caz, problema este că biblioteca grafică WinBGIm este realizată pe 32 biți, pe când compilatorul vostru este pe 64 biți. În acest caz, va trebui să faceți astfel:

Descărcați biblioteca grafică WinBGIm de aici: [WINBGIM.ZIP](#).

Atenție: pe web puteți găsi și alte variante ale acestei arhive, dar e posibil să fie nefuncționale.

După ce descarcăți și scoateți fișierele din această arhivă, veți face următoarele lucruri:

1. Descărcați compilatorul pe 32 de biți de la adresa următoare: <https://jmeubank.github.io/tdm-gcc/>
2. Veți folosi acest compilator, pentru a putea fi recunoscută biblioteca **WinBGIm**. Mergeți în CodeBlocks la **Settings->Compiler->Toolchain Executables** și scrieți calea către noul compilator: (de exemplu: **C:\TDM-GCC-32**)
3. Apoi la **Settings->Compiler->Linker Settings** scrieți în caseta din dreapta (**Other Linker Options**) următoarele: **-lbg -lgdi32 -lcomdlg32 -luuid -loleaut32 -lole32**
4. Faceți click pe OK.

- Extrageți fișierele **graphics.h**, **winmgim.h** și **libbgi.a** din arhivă.
- Copiați fișierele **graphics.h** și **winbgi.h** în folderul **include** din compilatorul TDM-GCC-32, care se găsește, cel mai probabil, aici: **C:\TDM-GCC-32\include**
- Copiați fișierul **libbgi.a** în folderul **lib** al directorului compilatorului MinGW, care se găsește, cel mai probabil, aici: **C:\TDM-GCC-32\lib**

Următorul videoclip explică foarte clar pașii pe care trebuie să-i urmați.

▼ CLICK AICI

02. [New] How to setup graphics.h in CodeBlocks v20.03 | ...



3. Dacă veți folosi sistemul de operare macOS sau Linux, cu CodeBlocks, atunci:

În locul bibliotecii grafice **WinBGIm** pentru Windows, încercați o bibliotecă grafică similară, cum ar fi **SDL_bgi**, pe care o găsiți aici: [SDLBGI](#).

Aceasta se bazează pe biblioteca grafică performantă **SDL**, dar adaptează funcțiile pentru a fi simple și compatibile cu biblioteca grafică **WinBGIm**.

4. Dacă veți folosi sistemul de operare Windows, cu Visual Studio (Visual C++), atunci:

Descărcați pachetul corespunzător de la adresa:

<https://home.cs.colorado.edu/~main/bgi/visual/>.

Puteți vedea și aceste videoclipuri, pentru ajutor:

1. <https://www.youtube.com/watch?v=5DN0T7YfyfM>
2. <https://www.youtube.com/watch?v=aRAvHudexYc>

Pentru orice alte probleme, apelați cu încredere la colegii voștri, pe serverul de Discord, în canalele speciale de voce și text referitoare la grafică.

Descrierea funcțiilor grafice

- Lista funcțiilor grafice, cu exemple la fiecare: <https://home.cs.colorado.edu/~main/cs1300/doc/bgi/index.html>
- Alte lucruri: <http://www.cs.colorado.edu/~main/cs1300/doc/bgi/bgi.html>.

Un exemplu de program de test pentru grafică este aici:

```
#include <iostream>
#include <graphics.h>
#include <winbgim.h>

int main()
{
    // int gd = DETECT, gm;
    // initgraph(&gd, &gm, "");
    initwindow(800,600); int x = 400, y = 300, r;
    for (r = 25; r <= 125; r += 20)
        circle(x, y, r);
    getch(); closegraph(); return 0;
}
```

Dacă acesta funcționează, puteți continua să folosiți orice funcție grafică și de lucru cu mouse-ul din biblioteca **WinBGIm**.

2. Exemple de programe

1. Exemplul 1: Reprezentări grafice simple

- Desenarea unei table de joc
- Desenarea unui poligon regulat
- Desenarea unei stele regulate

2. Exemplul 2: Citiri și scrieri în modul grafic- detalii și sugestie de rezolvare se găsesc la pagina 209 din cartea „Învățați limbajul Pascal în 12 lecții”:

[click aici](#)

3. Exemplul 3: Trasarea graficului unei funcții - detalii și sugestie de rezolvare se găsesc la pagina 214 din cartea „Învățați limbajul Pascal în 12 lecții”:

[click aici](#)

4. Exemplul 4: Trasarea bioritmului unei persoane - detalii și sugestie de rezolvare se găsesc la pagina 142 din cartea „Aplicații în C și C++”: [click aici](#)

5. Exemplul 5: Jocul pătratelor glisante - detalii și sugestie de rezolvare se găsesc la pagina 138 din cartea „Aplicații în C și C++”: [click aici](#)

6. Exemplul 6: Formarea imaginilor în lentilele convergente - detalii și sugestie de rezolvare se găsesc la pagina 135 din cartea „Aplicații în C și C++”: [click aici](#)

Atenție! Urmăriți mai întâi videoclipurile de mai jos:

- Exemplu simplu de utilizare a graficii: [click aici](#)
- Desenarea unui poligon regulat: [click aici](#)
- Desenarea unei stele: [click aici](#)
- Figuri recursive: [click aici](#)
- Graficul unei funcții (21:33): [click aici](#)

3. Desenarea unei stele regulate

Definiți o structură de date adecvată pentru a reprezenta o stea regulată cu un număr de colțuri (cel puțin 3), cu marginea de o anumită culoare și grosime, cu o anumită culoare în interior, cu o anumită mărime și o anumită „deschidere” a colțurilor, cu o anumită amplasare sub un anumit unghi față de orizontală a primului colț. Se pot folosi funcțiile trigonometrice sin și cos, din math.h. Scrieți o funcție care să deseneze o asemenea stea, oarecare, amplasată oriunde și oricum, de culoare oarecare etc.

▼ Click aici pentru o soluție, dar care trebuie adaptată la biblioteca grafică.

```
#include <iostream>
#include <math.h>

using namespace std;

// programul poate fi testat in DevC++, instaland biblioteca grafica WinBGIm
// sau in Processing 3.0
// folosind functiile grafice specifice

// presupunem ca dispunem de urmatoarele functii grafice

void setcolor(unsigned int c)
{
    // stabileste culoarea pentru a desena urmatoarele linii,
```

```

    // pana la schimbarea culorii de un alt apel al acestei functii (initial se considera c=0, implicit);
    // c este un cod de culoare, care corespunde unei valori între 0 si 15, de exemplu 0=negru, 1=albastru, 2=rosu etc.
}

void setwidth(unsigned int g)
{
    // stabileste grosimea (1<=g<=10, implicit g=1) de desenare a
    // a urmatoarelor linii, pana la un nou apel al acestei functii;
}

void line(unsigned int x1, unsigned int y1, unsigned int x2, unsigned int y2)
{
    // deseneaza/traseaza o linie între punctul de coordonate (x1, y1)
    // si cel de coordonate (x2, y2), in culoarea curenta, de grosimea curenta,
    // adica cele stabilite de functiile de mai sus
}

void setfillcolor(unsigned int c)
{
    // stabileste culoarea de umplere pentru urmatoarele apeluri ale functiei fill
    // de mai jos; c este implicit 0 (negru), dar poate fi orice valoarea 0<=c<=15;
}

void fill_(unsigned int x1, unsigned int y1, unsigned int c)
{
    // coloreaza (prin umplere) in culoarea stabilita de functia setfillcolor,
    // conturul inchis, marginit de o linie poligonala inchisa, avand culoarea c;
    // colorarea incepe in punctul de coordonate (x1, y1), care se presupune a fi
    // in interiorul acelei linii poligonale.
}

struct stea
{
    int xCentru, yCentru; // xc si yc sunt coordonatele centrului stelei,
    // adica ale cercurilor circumscrise celor doua poligoane regulate, formate din colturile stelei,
    // respectiv din punctele interioare
    int nrColturi; // nr de colturi ale stelei
    int raza; // raza cercului mare (unde se afla colutile stelei
    float coef; // un numar subunitar, reprezentand raportul dintre raza mica
    // si raza mare (acest coeficient determina automa cat de "deschisa"
    // sau de "culturoasa" este steaua
    float unghiRotire; // unghiul dintre dreapta Ox si
    // dreapta formata din centrul stelei si primul colt al stelei
    int culoareMargine; // culoarea marginii
    int grosimeMargine; //grosimea marginii
    int culoareStea; // culoarea interiorului stelei, fara margine
};

void deseneaza(stea Stea)
{
    // folosim variabile locale, cu denumiri scurte, le initializam
    int xc=Stea.xCentru;
    int yc=Stea.yCentru;
    int n=2*Stea.nrColturi;
    int r=Stea.raza;
    float gama=Stea.unghiRotire;
    // calculam raza pentru cercul format de punctele interioare stelei
    int r2=r*Stea.coef;
    // stabilim elementele de desenare
    // setcolor(Stea.culoareMargine); setwidth(Stea.grosimeMargine);
    // calculam unghiul facut de axa OX cu fiecare dintre punctele ce formeaza steaua (fie ca sunt colturi, fie puncte interioare)
    float alfa=2*3.1415926/n; // pi/n

    // folosim doi vectori pentru a pastra in ei coordonatele punctelor din stea
    // (exterioare si interioare)
    // se poate evita folosirea vectorilor,
    // prin memorarea doar a ultimelor doua puncte ale liniei poligonale
    // sau, daca e necesar, se pot memora coordonatele acestor puncte
    // chiar in definitia stelei

#define MAX_PUNCTE 100
    int x[MAX_PUNCTE];
    int y[MAX_PUNCTE];
    int i;
    for (i=0; i<=n; i++)
    {
        if (i%2!=0)
        {
            x[i]=int(xc+r2*cos(alfa*i+gama));
            y[i]=int(yc+r2*sin(alfa*i+gama));
        }
        else
        {
            x[i]=int(xc+r*cos(alfa*i+gama));
            y[i]=int(yc+r*sin(alfa*i+gama));
        }
    }
    for (i=0; i<=n-1; i++)
    {

```

```

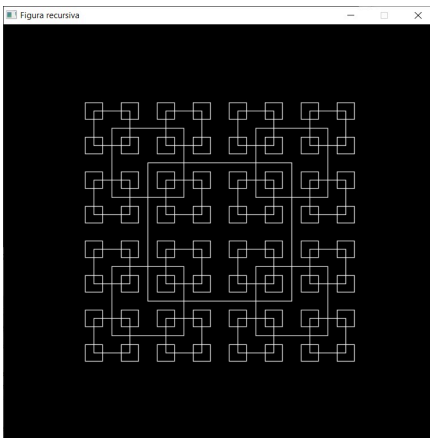
        line(x[i],y[i],x[i+1],y[i+1]);
    }
    // umplem steaua, plecand din centrul ei
    setfillcolor(Stea.culoareStea);
    fill_(Stea.xCentru,Stea.yCentru,Stea.culoareMargine);
}

int main()
{
    // exemplu de initializare a unei stele
    stea S;
    S.xCentru=450;
    S.yCentru=300; // coordonate centru stea
    S.nrColturi=5; // o stea cu 5 colturi
    S.unghiRotire=0.785; // aproximativ pi/4
    S.raza=200; // raza cercului format de colturi va fi 200;
    S.coef=0.5; // punctele interioare vor forma un cerc de raza jumătate din S.raza
    S.culoareMargine=2; // culoarea marginii va fi, de exemplu, rosu
    S.grosimeMargine=3; // grosimea marginii de 3 pixeli
    S.culoareStea=14; // culoarea stelei va fi, de exemplu, galben
    // exemplu program de test
    deseneaza(S);
    cout << "Atentie! Programul cu stea poate fi testat ";
    cout << "daca dispuneti de acele functii grafice!" << endl;

    return 0;
}

```

4. Figuri recursive cu cercuri si patrate



▼ CLICK AICI pentru exemple de cod

```

void patrat(int x, int y, int lat)
{
    line(x-lat/2,y-lat/2,x-lat/2,y+lat/2); line(x-lat/2,y+lat/2,x+lat/2,y+lat/2);
    line(x+lat/2,y+lat/2,x+lat/2,y-lat/2); line(x+lat/2,y-lat/2,x-lat/2,y-lat/2);
}
void diamantPatrat(int x, int y, int lat)
{
    if (lat>40)
    {
        patrat(x,y,lat);
        diamantPatrat(x-lat/2,y+lat/2,lat/2); diamantPatrat(x+lat/2,y+lat/2,lat/2);
        diamantPatrat(x+lat/2,y-lat/2,lat/2); diamantPatrat(x-lat/2,y-lat/2,lat/2);
    }
}
void diamantCerc(int x, int y, int lat)
{
    if (lat>40)
    {
        circle(x,y,lat);
        diamantCerc(x-lat/2,y,lat/2); diamantCerc(x,y+lat/2,lat/2);
        diamantCerc(x+lat/2,y,lat/2); diamantCerc(x,y-lat/2,lat/2);
    }
}

```

5. Trasarea graficului unei funcții oarecare

Ne propunem, în continuare, să realizăm graficul unei funcții oarecare $f: \mathbb{R} \rightarrow \mathbb{R}$ (definită în cadrul programului), într-o zonă dreptunghiulară (x_1, y_1, x_2, y_2) oarecare a ecranului, cu $y_1 > y_2$. Funcția va fi reprezentată pe un anumit interval $[a, b]$, deci x aparține lui $[a, b]$.

Se vor stabili, mai întâi, valorile minimă și maximă ale funcției, pe intervalul $[a, b]$. Să zicem că ele sunt y_{min} și y_{max} . Atunci vom face o transformare a imaginii reale a graficului în imaginea din zona de pe ecran a acestui grafic.

Această transformare va duce punctul $P(x, y)$ din planul real în punctul $Q(xe, ye)$ din planul ecranului calculatorului. De aceea vom utiliza două transformări liniare, care vor duce pe x în xe , iar pe y în ye . Funcțiile pe care le vom folosi vor avea forma $m \cdot x + n$.

Pentru a determina - pentru ambele transformări - pe m și n , va trebui să aplicăm formulele de transformare pentru capetele intervalelor. Așadar, având: $xe = m \cdot x + n$, obținem $x1 = m \cdot a + n$, respectiv $x2 = m \cdot b + n$, de unde se determină m și n , deci xe în funcție de x . La fel se determină ye , în funcție de y . În final, se vor obține formulele:

- $xe = (x2 - x1) \cdot x / (b - a) + (b \cdot x1 - a \cdot x2) / (b - a)$
- și $ye = (y2 - y1) \cdot y / (ymax - ymin) + (ymax \cdot y1 - ymin \cdot y2) / (ymax - ymin)$.

Valorile xe și ye vor fi rotunjite la numere întregi, deoarece în ecranul grafic se lucrează cu coordonate întregi și nu reale.

Video: Graficul unei funcții (21:33): [click aici](#)

Propunem să combinați programul care va reprezenta grafic o funcție cu unul care să verifice corectitudinea sintactică a unei funcții, precum și cu un program care să evalueze expresii.

Astfel, veți putea realiza un program care să reprezinte grafic orice funcție, a cărei expresie să fie dată de la tastatură. În loc ca funcția să fie parte componentă din codul programului, ar apărea o funcție C/C++ care să evalueze o funcție matematică într-un punct oarecare, funcția matematică fiind exprimată ca un șir de caractere.

6. Trasarea grafică a bioritmului unei persoane

Specialiști din diferite domenii ale științelor care au ca obiect de studiu omul, printre care și medicina, au ajuns la concluzia că puterea fizică, psihică și intelectuală a oricărei persoane evoluează funcțional sub forma unor sinusoidale, ale căror perioade sunt de 23, 28 și 33 de zile, iar începutul sinusoidelor depinde de data nașterii; pentru o persoană născută pe 1 ianuarie în anul 1, se consideră că bioritmul său pornește cu cele trei curbe de la 0 (medie), în sens crescător. Pornind de la ideile de mai sus, prezentăm în continuare un simplu program de trasare grafică a bioritmului pentru o anumită persoană. Programul cere utilizatorului să introducă data sa de naștere, numele său, precum și data de referință.

Bioritmul va fi trasat pe o perioadă de 32 de zile, începând cu data de referință stabilită.

▼ CLICK AICI pentru un cod sursă

```
#include <stdio.h>
#include <graphics.h>
#include <math.h>
#include <stdlib.h>
#include <dos.h>
#define pi 3.1415926

void OpenGraph()
{
    /* int gd=0, gm; initgraph(&gd,&gm,"C:\\BC\\BGI"); */
    initwindow(640,480,"Bioritm");
}

int anBisect(int y)
{
    if (y%400==0 || y%4==0 && y%100!=0)
        return 1;
    else
        return 0;
}

int NrZile(int d, int m, int y)
{
    int n;
    const int nn[12]={0,31,59,90,120,151,181,212,243,273,304,334};
    n=nn[m-1];
    n=n+365*y+y/4 + d + 1;
    n=n-y/100 + y/400;
    if (m>2 && !anBisect(y)) n--;
    return n;
}

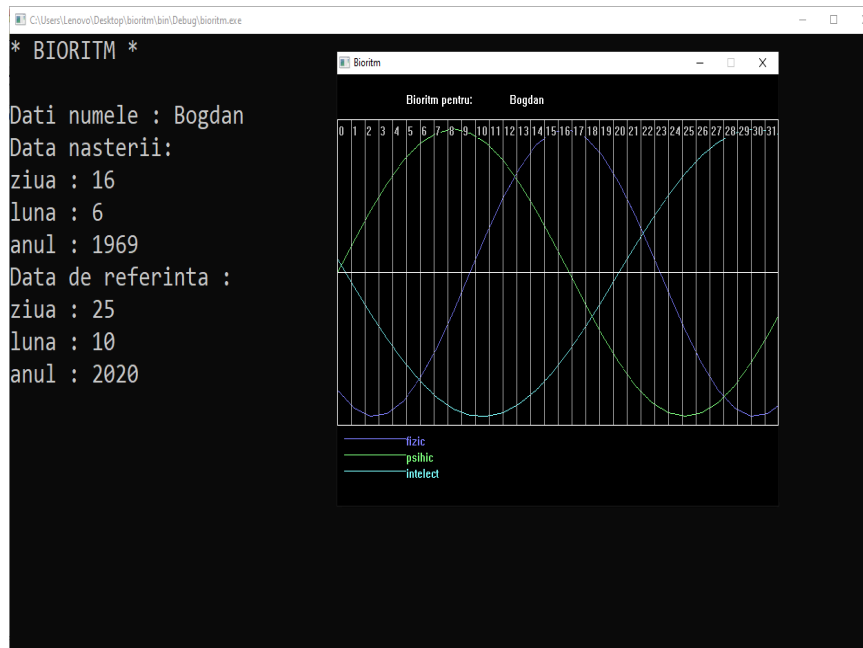
void Inputt(char *sir, int *nr)
{
    printf("%s",sir);
    scanf("%d",nr);
}

int main()
{
    int p[3] = {23,28,33}; int q[3];
    int m1,d1,y1,m2,d2,y2,i,n,nz,se,sen;
    float ur,sr; char nume[40]; fflush(stdin);
    printf("* BIORITM *\n\n");
    printf("Dati numele : "); gets(nume); printf("Data nasterii:\n"); Inputt("ziua : ",&d1); Inputt("luna : ",&m1); Inputt("anul : ",&y1);
    printf("Data de referinta :\n"); Inputt("ziua : ",&d2);
    Inputt("luna : ",&m2); Inputt("anul : ",&y2);
    nz=NrZile(d1,m1,y1);
    nz=NrZile(d2,m2,y2)-nz;
    OpenGraph();
    outtextxy(100,20,"Bioritm pentru: "); outtextxy(250,20,nume);
    for (i=0; i<3; i++)
    {
        setcolor(9+i); line(10,405+18*i,100,405+18*i);
```

```

    if (i==0) outtextxy(100,400+18*i,"fizic");
    else if (i==1) outtextxy(100,400+18*i,"psihic");
    else outtextxy(100,400+18*i,"intelect");
    q[i]=nz % p[i]; ur=2*pi*q[i]/p[i];
    sr=sin(ur); se=(sr+1)*80;
    for (n=nz; n<=nz+31; n++)
    {
        q[i] = n % p[i]; ur = 2*pi*q[i]/p[i];
        sr=sin(ur); sen=(sr+1)*160;
        line(24*(n-nz-1),480-(100+se),24*(n-nz),480-(100+sen));
        se=sen;
        delay(30);
    }
}
setcolor(7);
char nr_zi[2];
for (i=0; i<=31; i++)
{
    itoa(i, nr_zi, 10);
    line(20*i,50,20*i,390);
    outtextxy(20*i+2,55,nr_zi);
    delay(30);
}
line(getmaxx(),50,getmaxx(),390);
setcolor(15);
line(0,220,getmaxx(),220);
rectangle(0,50,getmaxx(),390);
getch();
closegraph();
}

```



Funcția `NrZile` determină numărul de zile scurse de la 1 ianuarie anul 1, până la o anumită dată. Astfel se poate determina numărul de zile între data de naștere a persoanei și data de referință. Programul poate fi rescris în orice altă implementare de C și pe orice alt tip de calculator, însă de fiecare dată trebuie să se țină cont de numărul de pixeli din suprafața de desenare și de sistemul de axe xOy. În cazul nostru, dimensiunea ecranului a fost de 640×480 pixeli, iar orientarea axei Oy a fost de sus în jos, iar a axei Ox de jos în sus. Originea axelor este în colțul din stânga-sus a zonei de desenare. Deși calculatorul nu greșește, e posibil ca oamenii de știință care au inventat bioritmul să fi greșit de la bun început, așa încât rămâne la latitudinea voastră dacă veți lua în considerare rezultatele furnizate de acest program sau nu, tratând programul ca un simplu amuzament.

7. Jocul pătratelor glisante

Un joc cu strategie câștigătoare "Jocul pătratelor glisante / alunecătoare", pe care vi-l prezentăm în continuare, poate constitui o problemă interesantă de programare pentru orice student. Pe o tablă cu $n \times n$ pătrățele (n întreg impar) sunt așezate, alternativ („în șah”), piese galbene și roșii, lăsând pătrățelul din mijlocul tablei neocupat.

Cei doi jucători mută, alternativ, ortogonal, câte o piesă proprie alăturată spațiului liber, în spațiul liber.

Pierde jucătorul care nu mai poate muta (deci spațiul liber este înconjurat numai de piese ale adversarului, eventual și de marginea tablei).

Se poate arăta ușor că al doilea jucător are o strategie sigură de câștig, adică, dacă urmează un anumit algoritm, poate câștiga tot timpul, indiferent de ce mută primul jucător.

Pentru a explica strategia câștigătoare, să „acoperim” tabla de joc cu „dominouri” (dreptunghiuri formate din două pătrățele elementare), lăsând căsuța din mijloc neacoperită. Adică să „legăm” fiecare căsuță a tablei de o altă căsuță alăturată, dar fiecare căsuță aparține unei singure legături. Iar căsuța din centru să nu fie legată de nicio căsuță.

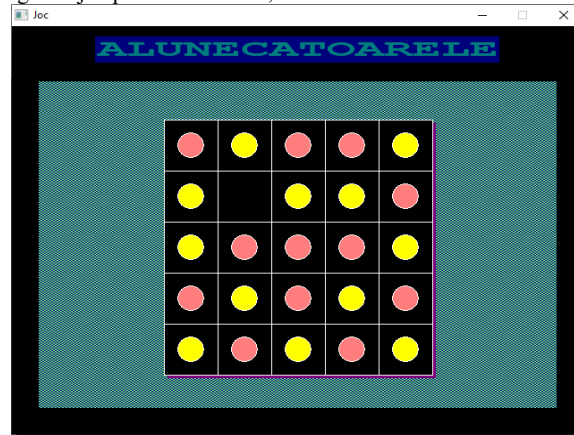
Vom observa că primul jucător, după prima mutare, va ocupa centrul și va elibera unul din capetele unui domino. Astfel, cel de al doilea jucător va putea să mute piesa din celălalt capăt al dominoului. Deci, în orice moment, cel de al doilea jucător poate muta. Partida nu se poate termina remiză, deci primul jucător

va pierde.

O metodă generală de aranjare a dominourilor, pe o tablă oarecare $n \times n$, cu n impar, este ca dominourile să se așeze în spirală, pornind din colțul stânga-sus și ajungând în centrul tablei.

Această aranjare a dominourilor pe tablă va genera un vector **Opus**, cu semnificația: $\text{Opus}[i]=j \Rightarrow$ dacă pătrățelul i este liber, atunci jucătorul al doilea va muta piesa din pătrățelul j . Pătrățelele sunt numerotate de sus în jos, de la stânga la dreapta. Astfel, căsuța de pe linia i și coloana j va avea numărul $(n-1) * i + j$. De observat că și $\text{Opus}[j]=i$.

Programul următor implementează această strategie de joc pentru calculator, în care calculatorul este întotdeauna al doilea jucător și va câștiga întotdeauna.



O posibilă implementare a acestui joc în C/C++ poate fi cea de mai jos.

▼ [CLICK AICI](#) pentru codul sursă

```
/* JoculPatratelorAlunecatoare */
#include <stdio.h>
/* #include <conio.h> */
#include <graphics.h>
#include <stdlib.h>
#include <dos.h>
typedef unsigned char byte;
typedef unsigned int word;
byte Leg[26]={0,6,7,8,5,4,1,2,3,10,9,12,11,0,15,14,21,22,19,18,25,16,17,24,23,20};
byte spatiu=0, lat=60, x0=110, y0=50;
byte is, js, Tabla[7][7];

void OpenGraph()
{
    /* int gd=DETECT,gm;
    initgraph(&gd,&gm,"c:\\bc\\bgi"); */
    initwindow(640,480,"Joc");
}

byte CasutaLibera()
{
    return (5*(is-1)+js);
}

void InitTabla()
{
    byte i,j;
    is=3;
    js=3;
    for (i=1; i<=5; i++)
        for (j=1; j<=5; j++)
            if ((i+j) % 2 != 0)
                Tabla[i][j]=LIGHTRED;
            else
                Tabla[i][j]=YELLOW;
    Tabla[is][js]=spatiu;
    for (i=0; i<=6; i++)
    {
        Tabla[0][i]=YELLOW;
        Tabla[6][i]=YELLOW;
        Tabla[i][0]=YELLOW;
        Tabla[i][6]=YELLOW;
    }
}

void DesPiesa(int i, int j)
{
    if ((i!=is) || (j!=js))
    {
        setcolor(WHITE);
        rectangle(x0+lat*i, y0+lat*j, x0+lat*i+lat, y0+lat*j+lat);
        setfillstyle(SOLID_FILL,Tabla[i][j]);
        fillellipse(x0+lat*i+lat/2, y0+lat*j+lat/2,lat/4, lat/4);
    }
    else
}
```

```

    {
        setfillstyle(SOLID_FILL, BLACK);
        bar(x0+lat*i, y0+lat*j, x0+lat*i+lat, y0+lat*j+lat);
        setcolor(WHITE);
        rectangle(x0+lat*i, y0+lat*j, x0+lat*i+lat, y0+lat*j+lat);
    }
}

void DesTabla()
{
    byte i, j;
    setfillstyle(SOLID_FILL, MAGENTA);
    bar(x0+lat+4, y0+lat+4, x0+lat*6+4, y0+lat*6+4);
    for (i=1; i<=5; i++)
        for (j=1; j<=5; j++)
        {
            setfillstyle(SOLID_FILL, 0);
            bar(x0+lat*i, y0+lat*j, x0+lat*i+lat, y0+lat*j+lat);
            DesPiesa(i, j);
        }
}

void MutaCalc()
{
    byte isv, jsv, cas;
    isv=is;
    jsv=js;
    cas=Leg[Casutalibera()];
    is=(cas-1)/5+1;
    js=(cas-1)%5+1;
    Tabla[is][js]=spatiu;
    DesPiesa(is, js);
    Tabla[isv][jsv]=YELLOW;
    DesPiesa(isv, jsv);
}

void MutaOm()
{
    int tasta, mut;
    do
    {
        tasta=getch();
        if (tasta==0)
            tasta=getch();
        mut=0;
        switch (tasta)
        {
            case 75:
                if (is<5)
                {
                    if (Tabla[is+1][js]==LIGHTRED)
                    {
                        is++;
                        mut=1;
                        Tabla[is][js]=spatiu;
                        DesPiesa(is, js);
                        Tabla[is-1][js]=LIGHTRED;
                        DesPiesa(is-1, js);
                    }
                }
                break;
            case 77:
                if (is>1)
                {
                    if (Tabla[is-1][js]==LIGHTRED)
                    {
                        is--;
                        mut=1;
                        Tabla[is][js]=spatiu;
                        DesPiesa(is, js);
                        Tabla[is+1][js]=LIGHTRED;
                        DesPiesa(is+1, js);
                    }
                }
                break;
            case 80:
                if (js>1)
                {
                    if (Tabla[is][js-1]==LIGHTRED)
                    {
                        js--;
                        mut=1;
                        Tabla[is][js]=spatiu;
                        DesPiesa(is, js);
                        Tabla[is][js+1]=LIGHTRED;
                        DesPiesa(is, js+1);
                    }
                }
                break;
            case 72:
                if (js<5)
                {
                    if (Tabla[is][js+1]==LIGHTRED)
                    {
                        js++;
                        mut=1;
                        Tabla[is][js]=spatiu;
                        DesPiesa(is, js);
                    }
                }
                break;
        }
    }
}

```

```

                Tabla[is][js-1]=LIGHTRED;
                DesPiesa(is,js-1);
            }
            break;
        case 27:
        {
            closegraph();
            printf("Ati abandonat...");
            exit(1);
        }
    }
    /* if (!mut)
    {
        sound(200);
        delay(50);
        nosound();
    }
    */
}
while (!mut);
}

int EsteGata()
{
return ((Tabla[is-1][js]==YELLOW) && (Tabla[is+1][js]==YELLOW)
        &&(Tabla[is][js-1]==YELLOW) && (Tabla[is][js+1]==YELLOW));
}

/*void Sunet()
{
    byte i;
    for (i=1; i<=25; i++)
    {
        sound(300+20*i);
        delay(30);
    }
    nosound();
}
*/
int main()
{
    OpenGraph();
    setbkcolor(BLUE);
    setfillstyle(9,LIGHTGREEN);
    bar(30,65,getmaxx()-30,getmaxy()-30);
    settextstyle(DEFAULT_FONT,HORIZ_DIR,4);
    settextjustify(CENTER_TEXT,CENTER_TEXT);
    setcolor(CYAN);
    outtextxy(320,35,"ALUNECATOARELE");
    InitTabla();
    DesTabla();
    do
    {
        MutaOm();
        delay(200);
        MutaCalc();
    }
    while (!EsteGata());
    outtextxy(320,100,"AI PIERDUT.");
    /* Sunet(); */
    getch();
    closegraph();
    printf("Iarasi ati pierdut ...");
    return 0;
}

```

Pentru utilizare mouse in joc se poate folosi:

```

void utilizareMouse()
{
    int linia,coloana,x,y;
    int x1, y1, x2, y2;
    int xmioloc, ymioloc;
    if(ismouseclick(WM_LBUTTONDOWN))
    {
        clearmouseclick(WM_LBUTTONDOWN);
        x=mousex();
        y=mousey();
        if (!(x>=stanga && x<=stanga+width
            && y>=sus&&y<=sus+height))
            gata=true;
        else
        {
            linia=(y-sus)/latura+1;
            coloana=(x-stanga)/latura+1;
            if (TablaDeJoc[linia][coloana]==0)
            {
                TablaDeJoc[linia][coloana]=1;
                // stergere veche
                // desenare noua
            }
        }
    }
}

```

```
}  
}
```

8. Alte probleme de grafică

Încercați să rezolvați problemele de la adresa: https://docs.google.com/document/d/1vtrwaRBMtBiL9FVDpXNJM7gpfiCqzM6GLVfwK13W_Cc/edit?usp=sharing

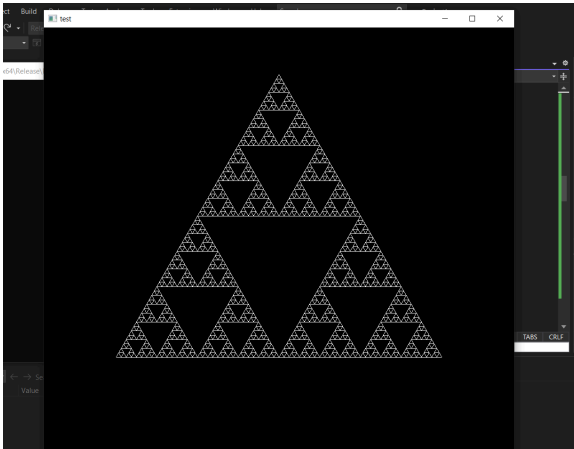
9. Un exemplu de utilizare a bibliotecii SFML in Visual Studio

(via Ioan Rosu, grupa A2 (2023-2024))

Puteti folosi acest tutorial pentru instalarea bibliotecii grafice SFML in Visual Studio, versiune cel putin din 2013:

<https://www.sfm1-dev.org/tutorials/2.6/start-vc.php>

Programul urmator deseneaza triunghiul lui Sierpinski.



```
#include <SFML/Graphics.hpp>  
#include <math.h>  
  
sf::RenderWindow window(sf::VideoMode(1000, 1000), "test");  
  
void drawTriangle(sf::Vector2f coords, float size)  
{  
    sf::Vertex line1[2], line2[2], line3[2];  
  
    sf::Vector2f point1, point2, point3;  
    point1.x = coords.x;  
    point1.y = coords.y - size;  
    point2.x = coords.x + size/2*sqrt(3);  
    point2.y = coords.y + size/2;  
    point3.x = coords.x - size/2*sqrt(3);  
    point3.y = coords.y + size/2;  
  
    line1[0] = point1;  
    line1[1] = point2;  
  
    line2[0] = point2;  
    line2[1] = point3;  
  
    line3[0] = point3;  
    line3[1] = point1;  
  
    window.draw(line1, 2, sf::Lines);  
    window.draw(line2, 2, sf::Lines);  
    window.draw(line3, 2, sf::Lines);  
}  
  
void triunghiSierpinski(sf::Vector2f coords, float size)  
{  
    if (size > 4)  
    {  
        drawTriangle(coords, size);  
  
        sf::Vector2f point1, point2, point3;  
        point1.x = coords.x;  
        point1.y = coords.y - size/2;  
        point2.x = coords.x + size/4*sqrt(3);  
        point2.y = coords.y + size/4;  
        point3.x = coords.x - size/4*sqrt(3);  
        point3.y = coords.y + size/4;  
    }  
}
```

```

        triunghiSierpinski(point1, size / 2);
        triunghiSierpinski(point2, size / 2);
        triunghiSierpinski(point3, size / 2);
    }
}

int main()
{
    sf::Vertex line[2] =
    {
        sf::Vertex(sf::Vector2f(10, 10)),
        sf::Vertex(sf::Vector2f(150, 150))
    };

    while (window.isOpen())
    {
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        window.clear(sf::Color::Black);

        triunghiSierpinski(sf::Vector2f(500, 500), 400);

        window.display();
    }

    return 0;
}

```